Soundness of Typed Transitions in the Linear π-Calculus

Technical Report
v1, 2 April 2026

Adrian Francalanza[1], Marco Giunti[2], and Antonio Ravara[3]

[1] Faculty of ICT, University of Malta
[2] Faculty of Science and Technology, University of Algarve
[3] NOVA LINCS and NOVA FCT

Abstract. We present a Coq mechanisation for a Labelled Transition System (*LTS*) with global information about resource visibility and capability. The LTS states are linear π-calculus processes, *i.e.*, concurrent programs that can dynamically create and share linear and unrestricted channels on other channels, paired with the type environment that typechecks them. The type environment is augmented with auxiliary information relating to the usage status and visibility (from an interacting process perspective). This global information determines what transitions are allowed. In turn, the transitions themselves update the global information as computation progresses. The mechanisation shows that the LTS (i) observes subject reduction for well-typed linear π-calculus processes, (ii) preserves Barendregt's convention for binders, and (iii) is deployed as a certified OCaml interpreter extracted from Coq. To the best of our knowledge, this is the first mechanisation of a typed LTS for the linear π-calculus.

Table of Contents

Soundness of Typed Transitions in the Linear π-Calculus [2em] Technical Report v1, 2 April 2026 1
Adrian Francalanza, Marco Giunti, and Antonio Ravara
1 Introduction 3
2 A linear π-calculus 4
2.1 Types and Type Environments 6
2.2 Type Algebra 7
2.3 Type system 8
3 Typed labelled transitions 10
4 Subject reduction, and other results 14
4.1 Subject Reduction Theorem 15
4.2 Other results 15
5 Conclusions 16
A Appendix 17
A.1 Mechanisation of Subject Reduction 18
Bibliography 20

1 Introduction

The linear π-calculus [24], by Kobayashi *et al.*, is an influential extension of the π-calculus [27] where channels are partitioned into either linear or unrestricted. The authors develop a type system to ensure that linear channels are used exactly once, and prove a subject reduction (*SR*) property that guarantees that such usage is invariant over reductions. The calculus has been studied for its computational properties [41,42] and has acted as a foundation for more expressive formalisms such as session types [8].

Despite its merits, the linear π-calculus lacks a mechanised *typed Labelled Transition System (LTS)* that permits compositional reasoning on typed computation. Such a typed LTS judgement would take the form below to encode the following information:

$$\Gamma \triangleright P \xrightarrow{\alpha} \Gamma' \triangleright P' \tag{1}$$

(*i*) environment Γ typechecks process P, *i.e.*, $\Gamma \vdash P$, (*ii*) process P interacts with some implicit observer process Q, denoted by the action α, (*iii*) the observer Q is itself *typed*, with respect to some implicit type environment Δ, *i.e.*, $\Delta \vdash Q$, where (*iv*) the two type environments are in agreement, *i.e.*, there exists some environment Λ that constitutes the composition of the two environment, $\Gamma + \Delta = \Lambda$, (*v*) after interacting with Q, process P transitions to a new state, P', that typechecks *wrt.* the (possibly different) environment Γ', *i.e.*, $\Gamma' \vdash P'$, and (*vi*) on its part, $\Delta \triangleright Q$ transitions on the dual of action α, *i.e.*, $\overline{\alpha}$, to a new state $\Delta' \triangleright Q'$ that also preserves typing.

Challenges. Several obstacles hinder the mechanisation of these behavioral frameworks in state-of-the-art proof assistants. For instance, binding constructs require proper handling of bound identifiers in order to model alpha conversion [3]; name scope extrusion further complicates this handling in the case of the π-calculus since the scope becomes dynamic. Another obstacle is the reliance on environment splitting assertions of the form $\exists \Gamma_1, \Gamma_2 . \Gamma = \Gamma_1 + \Gamma_2$ when defining transition rules for parallel processes [36,43]: the inherent underspecification of how these splitting are described (note the existential quantifications) complicates the mechanisation of proof searches. The idiomatic name scoping construct, $(\nu n)P$, though central to delineate locality, poses complications other than the handling of scope extrusion. For instance, a typed LTS based on the typing of [24], cannot be used to detect and locate deadlocks by *reporting offending scoped channels via their name*, since these are subject to *change* through alpha-conversion; in a reduction semantics, this mechanism is used to distinguish between the different instances of scoped names generated by replicated/recursive processes. Name scoping also makes it hard to model and reason about resource constrained computation where accounting for the number of (scoped and free) channels in use is required. For instance, the standard structural equivalence rule $(\nu n)P \equiv P$, valid whenever n is not a free name in P, can be used both to garbage collect redundant names but also to *introduce them arbitrarily*, since the rule is symmetric.

Proposed Solution. To this end, this paper proposes a linear π-calculus reformulation, for how free and scoped channel names are represented in the LTS states $\Gamma \triangleright P$ of Eq. (1): *all* channel names are recorded in the environment, Γ, to permit *global resource reasoning* on channels, and a *uniform mechanism to reference* them. In order to

still differentiate between global (*i.e.,* free) and local channels, *flags* are introduced in the environment mappings of Γ. Concretely, *visible* (`v`) and *hidden* (`h`) flags are used to correspond to free and local channels, respectively. Accordingly, the process P delimits locality via a hiding construct, $(\kappa\, n)P$, instead of the customary construct $(\nu n)P$, that has *no binding* functionality. To preserve locality and compositionality, the reformulation employs a third flag, *illegal* (`i`), which acts as the dual of the hidden flag for environment splitting, *i.e.,* $\mathtt{h} + \mathtt{i} = \mathtt{h}$. Typechecking across parallel processes can thus still be expressed in the Kobayashi *et al.* style of [24] using a (lifted) environment split. The flag annotations also permit the computation steps described in Eq. (1) to globally refer to hidden channels directly by their name, while also representing the hidden channels of the implicit observer Q (discussed earlier) via the (dual) illegal flag, `i`. The downside of our proposed mechanism is that the behaviour previously modelled via alpha-conversion needs to be explicitly managed by the LTS of Eq. (1).

Main contributions. 1. The definition of a compositional typed LTS for the linear π-calculus that manages names globally and book-keeps type information after transitions. The LTS does not employ (underspecified) environment splitting nor does it rely on structural equivalence, supporting a mechanisation leading to a certified interpreter. 2. The validation of the proposed typed LTS via a mechanised SR stating that if $\Gamma \vdash P$ then $\Gamma' \vdash P'$ as well. This is the first such result for a π-calculus relying on typed labelled semantics. 3. The mechanisation of the preservation of Barendregt's convention – bound variables are distinct from free ones and all binders bind variables not already in scope, a result [38,7] that here is a corollary of SR.

Outline. Sec. 2 introduces the language, the type algebra, and the type system, and illustrates their mechanisation in Coq. Sec. 3 presents the typed labelled semantics and its mechanisation. Sec. 4 contains the SR theorem; details of its mechanised proof are outlined in App. A. We conclude the section by showing how to extract a certified interpreter. Sec. 5 discusses related and future work, and the reusability of the techniques introduced herein. The paper contains links as *rooster images* that point to a public repository containing the *Coq development*; for instance 🐓 points to the proof of SR.[4]

2 A linear π-calculus

We employ a minimal version of the synchronous π-calculus, where replicated input is the only source of infinite behaviour. Instead of the usual name binding construct $(\nu n)P$, our calculus uses a Church-style hiding construct, $(\kappa\, n)P$, which does *not* assume alpha-conversion. The syntax given in Def. 1 assumes distinct enumerable sets for (channel) *names* $n, a, b, c, d \in$ Names, and *variables* $x, y, z \in$ Vars, and collectively refers to the set Names $\cup$ Vars as *identifiers*, ranged over by u, v. $N, L \subset$ Names denote *finite* sets of names and $M \Subset$ Names denote finite *multisets* of names, where $|M|$ converts M into a set by removing duplicates.

[4] We find it appropriate to use the old name of the Rocq Prover since we work with an older version (Coq 8.8.2).

Definition 1 (Process syntax).

$$P, Q, R \in \textsc{Proc} ::= \mathit{nil} \quad (\mathit{inert}) \quad | \; *u?x.P \quad (\mathit{replication}) \quad | \; P \| Q \quad (\mathit{composition})$$
$$| \; u?x.P \quad (\mathit{input}) \quad | \; u!v.P \quad (\mathit{output}) \quad | \; (\kappa\, n)P \quad (\mathit{hiding})$$

We often omit trailing nil and write $u!v$ in lieu of $u!v.\mathsf{nil}$. Our syntax only contains binders for variables, namely $u?x.P$ and $*u?x.P$ which have a static scope. The language does *not* employ name binders (whose scope is dynamic). We mimic the Barendregt's convention for names by only considering well-formed processes, Definition 2, where all hidden (restricted) names are disjoint from one another and also from visible (free) names. Formally, we let $\mathsf{hid}(P)$ be the multi-set of the hidden names of P where $\mathsf{hid}((\kappa\, n)P) = \mathsf{hid}(P) \uplus n$ ($\uplus$ being multi-set union); the remaining cases are homomorphic. In similar fashion, we let $\mathsf{vis}(P)$ be the multi-set of the names of P that are not hidden; the names of P are defined as $\mathbf{nm}(P) = \mathsf{hid}(P) \uplus \mathsf{vis}(P)$, and assume the predicate $\mathsf{noDup}(M) = \nexists n\,.\,\{n, n\} \Subset M$. The substitution of a *free* variable x of P with an identifier v, is noted as $P\{v/x\}$. Dynamic generation of "fresh" names at runtime (*cf.* § 3) is mimicked by the *relabelling of a hidden* name n with another name n' not occurring "elsewhere", noted $P\{|n'/n|\}$. The following paragraph clarifies this point.

Definition 2. *A process P is* well-formed *iff* $\mathsf{noDup}(\mathsf{hid}(P))$ *and* $\mathsf{hid}(P) \frown \mathsf{vis}(P) = \emptyset$.

Mechanisation of processes. We implement name and variables with type `name` (an alias for `nat`), thus avoiding any syntactic distinction among the two: this simplifies the treatment of type environments (*cf.* § 2.2). A semantic distinction is introduced by the mechanised type system (*cf.* § 2.3). Channels are implemented as *pairs of naturals* to allow us to track the replicated process body from where they are generated. See Rem. 1.

```
Notation name := nat.
Inductive dproc : Set := | dZero : dproc | dBang : name → name → name → name → dproc → dproc
  | dPar : dproc → dproc → dproc | dNu : name → name → dproc → dproc
  | dInp : name → name → name → name → dproc → dproc | dOut : name → name → name → name →
  dproc → dproc.
Lemma nat_pair_eq_dec : ∀ n m : name * name, {n = m} + {n ≠ m}.
```

We import `Coq.Lists.List` with `secLabs` providing *hidden channels* of processes (*cf.* hid).

```
Fixpoint secLabs (p : dproc) := match p with | dZero ⇒ nil | dPar p q ⇒ secLabs q ++ secLabs p
  | dBang p | dInp p | dOut p ⇒ secLabs p | dNu h i p ⇒ secLabs p ++ (h, i) :: nil end.
```

The substitution $P\{v/x\}$ is implemented as `pmap v1 v2 x1 x2`. Relabelling is only defined for processes that do not reuse a hidden channel twice (*cf.* Def. 2 and `List.NoDup`). Function `dep_relabel` receives a process, a multiset of channels, and *a proof* that the hidden channels of the process are unique, and returns a process where all hidden channels are relabelled by *shifting* the second entry of each channel with the maximal value among the second entries of the input multiset (*cf.* Rem. 1).

```
Fixpoint max (l : list (nat*nat)) : nat := match l with | nil ⇒ 0
  | (_, x) :: t ⇒ let m := max t in if x <? m then m else x end.
Fixpoint dep_relabel (p : dproc) (s : list (name*name)) : NoDup s → dproc :=
  match p with | dZero ⇒ fun _ ⇒ p
  | dBang x1 x2 y1 y2 p ⇒ fun pf : NoDup s ⇒ dBang x1 x2 y1 y2 (dep_relabel p s pf)
  | dInp x1 x2 y1 y2 p ⇒ fun pf : NoDup s ⇒ dInp x1 x2 y1 y2 (dep_relabel p s pf)
  | dOut x1 x2 y1 y2 p ⇒ fun pf : NoDup s ⇒ dOut x1 x2 y1 y2 (dep_relabel p s pf)
  | dPar p q ⇒ fun pf : NoDup s ⇒ dPar (dep_relabel p s pf) (dep_relabel q s pf)
  | dNu h i p ⇒ fun pf : NoDup s ⇒ dNu h (i + max s) (pmap (dep_relabel p s pf) h i h (i + max s))
  end.
```

We use function `dep_relabel` in the definition of the *typed labelled transition system* of processes defined in § 3.

Remark 1. Representing channels as pairs of `nat` is a design choice that yields two advantages. First, it allows us to generate fresh names (*i.e.,* globally unique) via `dep_relabel` in a *local manner* by simply incrementing the second element: starting with a set of unique pairs of the form `n 0`, any k increments to the second element will yield a unique pair `n k`. Second, by maintaining the first element `n` constant, we are able to retain the provenance of a dynamically generated channel; during bug reporting, this allows us to relate errors detected on dynamic code back to the channels specified in the static code.

2.1 Types and Type Environments

Types T in the universe Typ include channel types of the form $p[T]^m$, where $p \in$ Pol describes the channel *input* and *output* polarities, and $m \in$ Mul describes *linear* or *unrestricted* multiplicities. We abstract over base types considering a single one, bt.

Definition 3 (Type syntax).

$$T,S ::= \mathsf{bt} \quad (\textit{base}) \qquad | \; p[T]^m \quad (\textit{channel}) \qquad m ::= \omega \quad (\textit{unrestricted}) \qquad | \; \iota \quad (\textit{linear})$$
$$p,q ::= \updownarrow \quad (I/O) \qquad | \; \emptyset \quad (\textit{empty}) \qquad | \downarrow \quad (\textit{input}) \qquad | \uparrow \quad (\textit{output})$$

Type environments, ranged over by Γ and Δ, map identifiers to pairs $(\mathtt{f}, T)$, where $\mathtt{f} \in \{\mathtt{v}, \mathtt{h}, \mathtt{i}\}$ is a *flag*. The notation $\Gamma\backslash_a$ refers to Γ_1 whenever $\Gamma = \Gamma_1, a\colon (\mathtt{f}, T)$. Without name scoping, every Γ typing a process P (*cf.* § 2.3) includes *all* the names of P, $\mathbf{nm}(P)$ in its domain. To compensate, flags capture the visibility of an identifier in the process being typed: *visible* (`v`), *hidden* (`h`), or *illegal* (`i`). We refer to channels typed using linear (*resp.* unrestricted) channel types as linear (*resp.* unrestricted) channels, irrespective of their visibility status.

Mechanisation of type environments. The representation of types in Coq follows Def. 3. Channel types are produced with constructor `chan`; type bt is renamed `Top`. The flags `vis`, `hid`, and `ill` deploy `v`, `h`, and `i`, respectively. Type environments are represented as lists of type `typeEnv` (*cf.* Rem. 2). In order to preserve their semantics, we use the predicate `isMap` asserting that a type environment implements a partial function.

```
Inductive pol : Set := empty | inp | out | io.
Inductive mode : Set := lin | un.
Inductive piType : Set := | Chan : mode → pol → piType → piType | Top : piType.
Inductive flag : Set := hid | ill | vis.
Definition typeEnv := list ((name * name) * flag * piType).
Definition isMap (g : typeENv) := ∀ x f1 t1 f2 t2, In (x, f1, t1) g ∧ In (x, f2, t2) g →
  f1 = f2 ∧ t1 = t2.
```

The *maximum* of a type environment Γ, is the maximal second entry in the domain of Γ. This is the threshold that is used by function `dep_relabel` to perform the shifting.

```
Fixpoint domain (g: typeEnv) : list (nat * nat) :=
  match g with | (x, _, _) :: t ⇒ x :: domain t | nil ⇒ nil end.
Definition maximum (g : typeEnv) := max(domain(g)).
```

Type Operations

$$\emptyset \uplus p = p \qquad \downarrow \uplus \uparrow = \updownarrow \qquad \updownarrow \sqcup p = \updownarrow \qquad \emptyset \sqcup p = p \qquad \downarrow \sqcup \downarrow = \downarrow \qquad \uparrow \sqcup \uparrow = \uparrow \qquad \downarrow \sqcup \uparrow = \updownarrow$$

$$\mathsf{bt}+\mathsf{bt} = \mathsf{bt} \qquad p[T]^\iota + q[T]^\iota = p \uplus q[T]^\iota \qquad p[T]^\omega + q[T]^\omega = p \sqcup q[T]^\omega$$

$$\mathsf{bt} - \mathsf{bt} = \mathsf{bt} \qquad p[T]^m - \emptyset[T]^m = p[T]^m$$

$$p[T]^\iota - p[T]^\iota = \emptyset[T]^\iota \qquad \updownarrow [T]^\iota - \downarrow [T]^\iota = \uparrow [T]^\iota \qquad \updownarrow [T]^\iota - \uparrow [T]^\iota = \downarrow [T]^\iota$$

$$p[T]^\omega - p[T]^\omega = p[T]^\omega \qquad \updownarrow [T]^\omega - \downarrow [T]^\omega = \updownarrow [T]^\omega \qquad \updownarrow [T]^\omega - \uparrow [T]^\omega = \updownarrow [T]^\omega$$

Flag Combination

$$\mathtt{h} + \mathtt{i} = \mathtt{h} = \mathtt{i} + \mathtt{h} \qquad \mathtt{v} + \mathtt{v} = \mathtt{v} \qquad \mathtt{i} + \mathtt{i} = \mathtt{i}$$

Environment Splitting Relation

$$\emptyset + \emptyset = \emptyset \qquad \frac{\Gamma_1 + \Gamma_2 = \Gamma_3 \qquad (T = \mathsf{bt} \ \vee \ T = p[S]^\omega)}{(\Gamma_1, u{:}(\mathtt{f}, T)) + (\Gamma_2, u{:}(\mathtt{g}, T)) = \Gamma_3, u{:}(\mathtt{f}+\mathtt{g}, T)}$$

$$\frac{\Gamma_1 + \Gamma_2 = \Gamma_3}{(\Gamma_1, u{:}(\mathtt{f}, p[T]^\iota)) + (\Gamma_2, u{:}(\mathtt{g}, q[T]^\iota)) = \Gamma_3, u{:}(\mathtt{f}+\mathtt{g}, p \uplus q[T]^\iota)}$$

Fig. 1. Operations on Types and Type Environments

Remark 2. Type environments are represented as lists, rather than partial functions,*e.g.*, [4], because of the definition of the splitting relation $\Gamma_1 + \Gamma_2 = \Gamma$ of Fig. 1. The syntactic manipulation of flags and types in type environments with equal domain (*cf.* § 2.2) is the core mechanism deployed by environment splitting. This kind of transformation is better served by data structures rather than functions. Still, we pay the price of carrying dependent proofs establishing that the list representations do indeed abstract maps.

2.2 Type Algebra

Fig. 1 introduces an algebra on types based on the splitting (partial) operation $T_1 + T_2$. The operation is defined in terms of the polarity splitting of linear and unrestricted channels, which are treated differently, using, respectively, the symmetric operators $\uplus$ and $\sqcup$ (symmetric rules are elided): $\uplus$ is akin to set disjoint union (*i.e.,* a *partial* operation), whereas $\sqcup$ is akin to set union (*i.e.,* a *total* operation). Fig. 1 also defines a type subtraction operation; we abuse terminology and say that T is a *super-type* of S (*resp.* S is a subtype of T) when $T - S$ is defined. Environment splitting, $\Gamma_1 + \Gamma_2$, derived from Kobayashi *et al.* [24] is overloaded to handle visibility flags. Visible and illegal flags are idempotent, but hidden flags act as the commutative identity for the illegal flag. We follow [15] and require that split environments have the same domain.

Mechanisation of environment splitting. Each total operation on polarities, types, and flags of Fig. 1 is implemented as a `Fixpoint`, while partial operations in the same figure are deployed as relations defined with the command `Inductive`. We let `pol_cup` and

Consumed predicate

$$\frac{}{\mathtt{cons}(\emptyset)} \qquad \frac{\mathtt{cons}(\Gamma)}{\mathtt{cons}(\Gamma, u\colon (\mathtt{f}, \mathsf{bt}))} \qquad \frac{\mathtt{cons}(\Gamma)}{\mathtt{cons}(\Gamma, u\colon (\mathtt{f}, \emptyset[T]^{\iota}))} \qquad \frac{\mathtt{cons}(\Gamma)}{\mathtt{cons}(\Gamma, u\colon (\mathtt{f}, p[T]^{\omega}))}$$

Typing Rules

$$\textsc{tNil}\ \frac{\mathtt{cons}(\Gamma)}{\Gamma \vdash \mathsf{nil}} \qquad \textsc{tInU}\ \frac{\Gamma, u{:}(\mathtt{v}, p[T]^{\omega}), x{:}(\mathtt{v}, T) \vdash P \quad \downarrow \in p \quad \mathtt{cons}(\mathtt{proj}_{\mathtt{v},\mathtt{i}}(\Gamma))}{\Gamma, u{:}(\mathtt{v}, p[T]^{\omega}) \vdash *u?x.P}$$

$$\textsc{tIn}\ \frac{\Gamma, u{:}(\mathtt{v}, p[T]^{m} - \downarrow [T]^{m}), x{:}(\mathtt{v}, T) \vdash P \quad \downarrow \in p}{\Gamma, u{:}(\mathtt{v}, p[T]^{m}) \vdash u?x.P} \qquad \textsc{tNew}\ \frac{\Gamma, n{:}(\mathtt{v}, T) \vdash P}{\Gamma, n{:}(\mathtt{h}, T) \vdash (\kappa\, n)\,(P)}$$

$$\textsc{tPar}\ \frac{\Gamma_1 \vdash P_1 \quad \Gamma_2 \vdash P_2}{\Gamma_1 + \Gamma_2 \vdash P_1 \,\|\, P_2} \qquad \textsc{tOut}\ \frac{\Gamma, u{:}(\mathtt{v}, p[T]^{m} - \uparrow [T]^{m}), v{:}(\mathtt{v}, T' - T) \vdash P \quad \uparrow \in p}{\Gamma, u{:}(\mathtt{v}, p[T]^{m}), v{:}(\mathtt{v}, T') \vdash u!v.P}$$

Fig. 2. Linear Type System

`pol_plus` implement the polarity operators $\sqcup$ and $\uplus$, respectively, and `typ_plus` and `typ_remove` implement the type operators $+$ and $-$, respectively, and `flag_plus` implement the flag operator $+$. Predicate `flagOf g a f (typeOf g a t)` asserts that the flag (type) of channel a in the environment Γ is $\mathtt{f}$ (T). We use function `rm` to remove a channel n from a type environment Γ. The splitting $\Gamma_1 + \Gamma_2 = \Gamma_3$ of Fig. 1 is defined through the inductive predicate `tEnv_split`, written `tEnv_split g1 g2 g3 pf1 pf2 pf3`: each parameter $\mathtt{pf}_i$, $1 \leq i \leq 3$, is a proof that Γ_i is a map.

```
Inductive tEnv_split (g1 : typeEnv) (g2 : typeEnv) (g3 : typeEnv) (pf1 : isMap g1) (pf2 : isMap g2)
  (pf3 : isMap g3) : Prop :=
  | SpNil : g1 = nil → g2 = nil → g3 = nil → tEnv_split g1 g2 g3 pf1 pf2 pf3
  | SpLin : ∀ n f f' g p q p' t g''' pf' pf'' pf''', flag_plus f g f' → pol_plus p q p' →
    In (n, f, Chan lin p t) g1 → In (n, g, Chan lin q t) g2 → rm n g3 = g''' → flagOf g3 n f' →
    typeOf g3 n (Chan lin p' t) → tEnv_split (rm n g1) (rm n g2) g''' pf' pf'' pf''' →
    tEnv_split g1 g2 g3 pf1 pf2 pf3
  | SpUn : ∀ n f f' g p t g''' pf' pf'' pf''', flag_plus f g f' → In (n, f, Chan un p t) g1 →
    In (n, g, Chan un p t) g2 → tEnv_split (rm n g1) (rm n g2) g''' pf' pf'' pf''' → rm n g3 = g''' →
    flagOf g3 n f' → typeOf g3 n (Chan un p t) → tEnv_split g1 g2 g3 pf1 pf2 pf3
  | SpTop : ∀ n f f' g g''' pf' pf'' pf''', flag_plus f g f' → In (n, f, Top) g1 →
    In (n, g, Top) g2 → rm n g3 = g''' → flagOf g3 n f' →
    tEnv_split (rm n g1) (rm n g2) g''' pf' pf'' pf''' → typeOf g3 n Top →
    tEnv_split g1 g2 g3 pf1 pf2 pf3.
```

2.3 Type system

The type system is defined via the standard substructural typing rules in Fig. 2 using the type algebra of Fig. 1. It is a cosmetic variant of [24], and enforces the invariant that linear I/O capabilities are used exactly once through the predicate $\mathtt{cons}(\Gamma)$, to ensure that all linear permissions in a type environment are consumed. We let $\mathtt{proj}_{\mathtt{v},\mathtt{i}}(\emptyset) = \emptyset$, $\mathtt{proj}_{\mathtt{v},\mathtt{i}}(\Gamma, u\colon (\mathtt{f}, T)) = \mathtt{proj}_{\mathtt{v},\mathtt{i}}(\Gamma), u\colon (\mathtt{f}, T)$ when $\mathtt{f} \in \{\mathtt{v}, \mathtt{i}\}$, and $\mathtt{proj}_{\mathtt{v},\mathtt{i}}(\Gamma, u\colon (\mathtt{h}, T)) = \mathtt{proj}_{\mathtt{v},\mathtt{i}}(\Gamma)$. Note that whereas statically scoped variables add to the premise typing environment in rules tIn and tInU, hidden names are expected to be part of the conclusion type environment carrying the hidden $\mathtt{h}$ flag in rule tNew. All typing rules are syntax

directed. Rule TPAR poses the only source of under-specification when typechecking (bottom-up) for how to split the linear permissions across the environments Γ_1 and Γ_2.

Mechanisation of the typing rules. The predicate cons is implemented by the inductive type `cons`↯. The mechanisation of the rules follows. The predicate `typeCheck` receives an environment Γ and a proof that Γ satisfies `isMap`↯. To ease the explanation of the rules, we use the notation $\Gamma\backslash_n$ to indicate `rm g n`.

```
Inductive typeCheck (g : typeEnv) (pf : isMap g) : dproc → Prop :=
  | TNil :  cons g pf → typeCheck g pf dZero
  | TNew : ∀ p n t pf1, In (n, hid, t) g → typeCheck ((n, vis, t) :: (rm n g)) pf1 p →
    typeCheck g  pf (dNu (fst n) (snd n) p)
  | TPar : ∀ g1 g2 p1 p2 pf1 pf2, tEnv_split g1 g2 g pf1 pf2 pf → typeCheck g1 pf1 p1 →
    typeCheck g2 pf2 p2 → typeCheck g pf (dPar p1 p2)
  | TOutL : ∀ u p t v t' q t1 t2 pf1, type_remove (Chan lin p t) (Chan lin out t) t1 →
    type_remove t' t t2 → typeCheck ((u, vis, t1) :: (v, vis, t2) :: (rm v (rm u g))) pf1 q →
    In (u, vis, Chan lin p t) g → In (v, vis, t') g →
    typeCheck g pf (dOut (fst u) (snd u) (fst v) (snd v) q)
  | TOutUn : ∀ u p t v t' q t2 pf1, (p = io ∨ p = out) → type_remove t' t t2 →
    In (u, vis, Chan un p t) g → In (v, vis, t') g → typeCheck ((v, vis, t2) :: (rm v g)) pf1 q →
    typeCheck g pf (dOut (fst u) (snd u) (fst v) (snd v) q)
  | TInL : ∀ u p t x q t1 pf1, type_remove (Chan lin p t) (Chan lin inp t) t1 →
    In (u, vis, Chan lin p t) g → ¬ In x (domain g) → snd x < maximum g →
    typeCheck ((u, vis, t1) :: (x, vis, t) :: rm u g) pf1 q →
    typeCheck g pf (dInp (fst u) (snd u) (fst x) (snd x) q)
  | TInUn : ∀ u p t x q pf1, (p = io ∨ p = inp) → In (u, vis, Chan un p t) g → ¬ In x (domain g) →
    snd x < maximum g → typeCheck ((x, vis, t) :: g) pf1 q →
    typeCheck g pf (dInp (fst u) (snd u) (fst x) (snd x) q)
  | TInU : ∀ u p t x q pf1 pf2, (p = io ∨ p = inp) → In (u, vis, Chan un p t) g →
    ¬ In x (domain g) → snd x < maximum g → cons (vis_proj g ++ ill_proj g) pf2 →
    typeCheck ( (x, vis, t) :: g) pf1 q → typeCheck g pf (dBang (fst u) (snd u) (fst x) (snd x) q).
```

Case TNil accepts the inert process iff the predicate `cons`↯ holds. Case TNew applies when the process is $(\kappa\, n)\,(P)$, and channel n has flag $\mathtt{h}$ and type T in Γ, and $\Gamma\backslash_n, n\colon (\mathtt{v}, T)$ type checks P. Case TPar applies when we have a parallel process $P_1 \parallel P_2$ and environments Γ_1 and Γ_2 such that $\Gamma_1 + \Gamma_2 = \Gamma$ and $\Gamma_1 \vdash P_1$ and $\Gamma_2 \vdash P_2$. For each output (cf.TOUT) and input (cf.TIN) rule of Fig. 2 we have two versions corresponding to a linear or unrestricted channel subject. Case TOutL applies when we have an output process $u!v.Q$, and u is a linear channel of Γ. We require that: (1) both u and v are visible; (2) the subtraction T_1 of the output capability of u from the type of u is defined; (3) the subtraction T_2 of the payload type of u from the the type of v is defined; (4) the environment $\Gamma\backslash_u\backslash_v, u\colon (\mathtt{v}, T_1), v\colon (\mathtt{v}, T_2)$ type checks Q. Case TOutUn applies when we have an output process $u!v.Q$, and u is an unrestricted channel of Γ. We weaken the conditions (2) and (4) of TOutL: (2) u possesses the output capability; (4) the environment $\Gamma\backslash_v, v\colon (\mathtt{v}, T_2)$ type checks Q. The rules of the input $u?x.P$ and of the replicated input $*u?x.P$ follow (*cf.* TInL, TInUn, and TInU, respectively). In these rules, the variable x occurring as subject of the input is treated differently from a name, although it belongs to the same syntactic category (*cf.* name). In particular, in all these rules we require that: (1) u is visible; (2) the input capability of u with payload T is available; (3) *x is not used* in Γ and its second component is smaller than the maximum. The first part of (3) implements a "freshness' check that avoids clashing of variables: the assumption does not pose any limitation because we can always rename all clashing variables when typing processes at compile time. The second part of (3) is a *sanity check* that is needed to preserve the soundness of relabelling; App. A further discusses this point. Case TInL applies when we have an input process $u?x.Q$, and u is a linear channel of Γ: in addition to (1)–(3) above, we

Typed Transitions

$$\textsc{lOut}\ \frac{T'' = T' - T}{\Gamma, a{:}(\mathtt{v}, \uparrow [T]^{\iota}), n{:}(\mathtt{v}, T') \triangleright a!n.P \xrightarrow{a!n} \Gamma, a{:}(\mathtt{v}, \emptyset[T]^{\iota}), n{:}(\mathtt{v}, T'') \triangleright P}$$

$$\textsc{uRIn}\ \frac{\downarrow \in p \qquad T'' = T' + T \qquad \mathtt{f} \neq \mathtt{h} \qquad (\Gamma', P') = \mathsf{rlb}((\Gamma, c{:}(\mathtt{v}, p[T]^{\omega}), n{:}(\mathtt{f}, T')), P)}{\Gamma, c{:}(\mathtt{v}, p[T]^{\omega}), n{:}(\mathtt{f}, T') \triangleright *c?x.P \xrightarrow{c?n} \Gamma, c{:}(\mathtt{v}, p[T]^{\omega}), n{:}(\mathtt{v}, T''), \Gamma' \triangleright P[n/x] \parallel *c?x.P'}$$

$$\textsc{ParL}\ \frac{\Gamma \triangleright P_1 \xrightarrow{\alpha} \Gamma' \triangleright P_1'}{\Gamma \triangleright P_1 \parallel P_2 \xrightarrow{\alpha} \Gamma' \triangleright P_1' \parallel P_2} \qquad \textsc{Open}\ \frac{\Gamma, n{:}(\mathtt{v}, T) \triangleright P \xrightarrow{n'!n} \Gamma' \triangleright P'}{\Gamma, n{:}(\mathtt{h}, T) \triangleright (\kappa\, n)\,(P) \xrightarrow{n'!n} \Gamma' \triangleright P'}$$

$$\textsc{Hide}\ \frac{\Gamma, n{:}(\mathtt{v}, T) \triangleright P \xrightarrow{\alpha} \Gamma', n{:}\,(\mathtt{v}, S) \triangleright P' \quad n \notin \mathbf{nm}(\alpha)}{\Gamma, n{:}(\mathtt{h}, T) \triangleright (\kappa\, n)\,(P) \xrightarrow{\alpha} \Gamma', n{:}\,(\mathtt{h}, S) \triangleright (\kappa\, n)\,(P')}$$

$$\textsc{uComL}\ \frac{\begin{array}{c} S' = S - T \quad \Gamma, c{:}(\mathtt{v}, \updownarrow [T]^{\omega}), n{:}(\mathtt{v}, S) \triangleright P_1 \xrightarrow{c!n} \Gamma_1 \triangleright P_1' \\ \Gamma, c{:}(\mathtt{v}, \updownarrow [T]^{\omega}), n{:}(\mathtt{v}, S') \triangleright P_2 \xrightarrow{c?n} \Gamma_2, c{:}(\mathtt{v}, \updownarrow [T]^{\omega}), n{:}(\mathtt{v}, S) \triangleright P_2' \end{array}}{\Gamma, c{:}(\mathtt{v}, \updownarrow [T]^{\omega}), n{:}(\mathtt{v}, S) \triangleright P_1 \parallel P_2 \xrightarrow{\tau} \Gamma_2, c{:}(\mathtt{v}, \updownarrow [T]^{\omega}), n{:}(\mathtt{v}, S) \triangleright P_1' \parallel P_2'}$$

$$\textsc{lClsL}\ \frac{\begin{array}{c} S' = S - T \quad \Gamma, a{:}(\mathtt{v}, \uparrow [T]^{\iota}), n{:}(\mathtt{h}, S) \triangleright P_1 \xrightarrow{a!n} \Gamma_1 \triangleright P_1' \\ \Gamma, a{:}(\mathtt{v}, \downarrow [T]^{\iota}), n{:}(\mathtt{i}, S') \triangleright P_2 \xrightarrow{a?n} \Gamma, a{:}(\mathtt{v}, \emptyset[T]^{\iota}), n{:}(\mathtt{v}, S) \triangleright P_2' \end{array}}{\Gamma, a{:}(\mathtt{v}, \updownarrow [T]^{\iota}), n{:}(\mathtt{h}, S) \triangleright P_1 \parallel P_2 \xrightarrow{\tau} \Gamma, a{:}(\mathtt{v}, \emptyset[T]^{\iota}), n{:}(\mathtt{h}, S) \triangleright (\kappa\, n)\,(P_1' \parallel P_2')}$$

Fig. 3. Labelled Transition System (Selected Rules)

require that $\Gamma\backslash_u, u\colon T_1, x\colon (\mathtt{v}, T)$ types the continuation Q, where the subtraction T_1 of the input capability of u from the type of u must be defined. Case TInUn applies when we have an input process $u?x.Q$, and u is an unrestricted channel of Γ. We require that (4) $\Gamma, x\colon (\mathtt{v}, T)$ types Q. The last case, TInU, applies when we have a replicated input process $*u?x.P$ and u is an unrestricted channel of Γ. We require that (4) the projection $\natural$ of the visible and illegal names of Γ is consumed, and (5) $\Gamma, x\colon (\mathtt{v}, T)$ types Q.

3 Typed labelled transitions

Fig. 3 presents the operational semantics of our process language as a LTS over *configurations*: these are pairs of environments Γ, and processes, P, written as $\Gamma \triangleright P$, and constitute the states of our LTS. Similarly to previous work, e.g. [18,17], when writing $\Gamma \triangleright P$ we assume that *there exists* an environment that *type checks* P. However, in our setting the environment Γ_1 s.t. $\Gamma_1 \vdash P$ has *less or equal information* on the channels of P, *i.e.,* $\exists \Gamma_2.\ \Gamma = \Gamma_1 + \Gamma_2$; in contrast, Γ_1 has more or equal information in [18,17], *i.e.,* $\Gamma_1 <: \Gamma$. Note that, in our configurations $\Gamma \triangleright P$ channels are hidden (respectively visible) in Γ, only if they are hidden (respectively visible) in P.

The LTS is a ternary relation, where the first and third elements are a configuration and the second element is a label, ranged over by α. It is written as $\Gamma \triangleright P \xrightarrow{\alpha} \Gamma' \triangleright P'$ where α is either an internal (or idle) action τ, an input action $n?n'$, or an output action $n!n'$. The set of names of an action, $\mathbf{nm}(\alpha)$, are the empty if the action is idle and $\{n, n'\}$ otherwise. In the case of the input action, the judgement $\Gamma \triangleright P \xrightarrow{n?n'} \Gamma' \triangleright P'$ denotes that the configuration $\Gamma \triangleright P$ interacts with a (context) process, Q, that typechecks *wrt.* an environment Δ that can be composed with Γ, *i.e.,* $\Gamma + \Delta$ is defined. The transition therefore models the communication where $\Gamma \triangleright P$ inputs on channel n the name n' sent to it by the context $\Delta \triangleright Q$, to then evolve to P'. The interpretation of the output transition, $\Gamma \triangleright P \xrightarrow{n!n'} \Gamma' \triangleright P'$, is analogous. *Scope extruding* outputs, a characteristic feature of π-calculus LTSs, is described as an outputted name n' has visibility $\mathtt{h}$ in Γ; this also implies that the context interacted with must typecheck *wrt.* some environment Δ where the visibility of n' is $\mathtt{i}$, due to the requirement $\mathtt{h}+\mathtt{i}=\mathtt{h}$ from Fig. 1.

The LTS rules were designed along their mechanisation following a feedback-loop methodology where the high level definitions were tuned by considering the Coq implementation, and vice-versa (*e.g.,* to see how actions can do away with binding when expressing scope extrusion). The transition rules are *split-free*, *i.e.,* they do not employ the environment splits, $\Gamma_1 + \Gamma_2$. There are two reasons for this. First, this eliminates underspecification for derivations involving parallel processes, making the system more algorithmic: in § 4, we defend this thesis by presenting a certified OCaml interpreter extracted from the Coq code. Second, rule premises use the same environment for the most part (*e.g.,* ParL in Fig. 3), meaning that the implicit context with whom the process interacts is preserved for sub-derivations, in keeping with the intended interpretation of the transition. It is worth noting that, although transition derivations may start with a configuration $\Gamma \triangleright P$ at the root that is *typeable*, *i.e.,* $\Gamma \vdash P$, this need *not* be the case for configurations in the premises. Note also that processes in a transition are always *closed* (as opposed to the judgements of Fig. 2, where processes may contain free variables).

We have five rules for prefixes; here we cover two of them. Rule lOut allows an output process $a!n.P$ to transition with an output on a *linear* channel a and payload n (both *visible*) whenever the environment maps a to a linear channel type with *only* the output capability, $\uparrow$, to allow the observer to hold the dual input capability, $\downarrow$; the environment should also map n to a supertype of T, the declared payload type of a, captured by the condition $T''=T'-T$. The configuration transitions with action $a!n$ to reach a configuration where the environment has no capabilities left for a, and n has the remainder capability obtained by removing T from the prior type of n.

Rule uRIn describes the semantics of a replicated input $*c?x.P$ receiving a name n. Unlike the output case, the flag $\mathtt{f}$ of n is constrained, namely $\mathtt{f} \neq \mathtt{h}$, since hidden flags are reserved for hidden channels in P; whenever $\mathtt{f} = \mathtt{h}$, the observer can only be typed with flag $\mathtt{i}$ for n, and thus can never send the name n. The configuration reached always carries flag $\mathtt{v}$ for n. "Intrusion" of fresh names from the observer is expressed when the flag of n is $\mathtt{i}$ in the source configuration, meaning that the observer is typed with visibility $\mathtt{h}$ for n. The rule requires that (*i*) the environment possesses *at least* the input capability for c; (*ii*) the type of c does not change in the reached configuration. Replication comports that a transition labelled by $c?n$ spawns a *relabelled* copy P' of the input body P, to preserve the (encoding of) Barendregt's convention. The

definition of the operator rlb is axiomatic, and is mechanised by passing the domain of the environment to function `dep_relabel` introduced in Sec. 2; we avoid all details of the axiomatisation and refer to the paragraph on the mechanisation. Note, however, the addition of a *fresh environment* Γ' to the configuration transitioned to, containing the *new entries* needed to type check the hidden names of the relabelled continuation P'.

Rule Open models scope extrusion, removing a hiding construct $(\kappa\, n)P$ enveloping P by sending the name n as the payload of an output on some channel n'. We mimic the mechanism from the typing rule tNew of Fig. 2 and change the visibility of n from h to v in the premise, once the restriction is peeled off. Our framework does *not* rely on alpha-conversion, since it requires n to be in the type environment domain carrying the flag h. From the premise we (implicitly) know that the process transitioned to, namely P', does not restrict the name n and, accordingly, the configuration transitioned to in the conclusion must map n to a visible flag (v). Our formalism does not employ a dedicated syntax for the open output action (as found in [35,17]), since this information can be obtained from the flag of the payload in the environment. The other rule handling name restriction is Hide. It is analogous to rule Open in many respects, but (*i*) the action does not concern the name being restricted in the conclusion; (*ii*) the restriction construct (and hidden flag) is preserved in the configuration transitioned to.

We have five rules for parallel processes (apart from the symmetric ones); we only discuss three of them. Independent transitions across processes is described by rule ParL. Its conclusion permits parallel processes $P_1 \,\|\, P_2$ to transition whenever the (left) sub-process P_1 can transition with the *same* environment of the parallel composition, Γ. The environment of the configuration reached is that reached by the sub-process in the premise. This formulation avoids the need to split the environment while preserving the implicit observer with which the interaction occurs, namely some $\Delta \triangleright Q$ that is in agreement with $\Gamma \triangleright P_1 \,\|\, P_2$, *i.e.*, $\Gamma + \Delta$ is defined.

Rule uComL allows a communication of a visible channel on a unrestricted channel c, given that: (i) the environment of the premises retains the full capability on c; (ii) the reached environment differs from the starting environment by possibly extending the environment with new entries (*e.g.*, whenever the input action has been fired by using uRIn) The premises require that n is visible and that the output and input actions, $c!n$ and $c?n$, can be performed in the respective sub-processes with designated environments; these are obtained by providing the necessary conditions for the action by splitting the type of n accordingly using the object type of channel c (the type of channel c need *not* be split since it is unrestricted). The environment reached after the communication is the one reached by the input action premise, which takes into account the case where the input $c?n$ is the result of a replicated input that may extend the domain of the environment. Rule lClsL allows for sending a hidden channel n over a linear channel a, whenever the environment in the premises has the full capability of a. It requires that: (*i*) the premises split the capabilities for output and input; (*ii*) the premises split the flag of n, which is hidden, as the visibility h for the premise executing the output, and $\mathtt{i}$ for the premise executing the input; (*iii*) the environment reached after the transition is equal to the starting environment but for the type of a, where both (linear) capabilities are now consumed *i.e.*, channel a now has empty capabilities.

Mechanisation of LTS rules. The LTS in Fig. 3 is implemented by type `lts` which is defined using a proof that the domain of the initial environment Γ does not contain duplicates (*cf.* `List.NoDup`); this is stronger than `isMap(Gamma)` and is used with `List.In` to implement conditions of the form $\Gamma(u) = (\mathtt{f}, T)$, from now on written as `In (u, f, t) g`.

```
Inductive lts (g : typeEnv) : dproc → action → typeEnv → dproc → NoDup (domain g) → Prop :=
| parL : ∀ p1 a g1 q p2 pf, lts g p1 a g1 p2 pf → lts g (dPar p1 q) a g1 (dPar p2 q) pf
| lOut : ∀ p a1 a2 b1 b2 pf t t1 t2 g1, type_remove t1 t t2 → In ((a1, a2), vis, Chan lin out t) g→
  g1=(((a1, a2), vis, Chan lin empty t):: ((b1, b2), vis, t2):: (rm (a1,a2) (rm (b1,b2) g))) →
  In ((b1, b2), vis, t1) g → lts g (dOut a1 a2 b1 b2 p) (aOut a1 a2 b1 b2) g1 p pf
| uRin : ∀ pol p a1 a2 x1 x2 b1 b2 pf f t t1 t2 g1, pol = inp ∨ pol = io → type_plus t1 t t2 →
  f ≠ hid → In ((a1, a2), vis, Chan un pol t) g → In ((b1, b2), f, t1) g →
  let q := dep_relabel p (domain g) pf in
  g1 = (newEntries g (secLabs q) (maximum g)) ++(((b1, b2), vis, t2) :: rm (b1, b2) g) →
  lts g (dBang a1 a2 x1 x2 p) (aIn a1 a2 b1 b2) g1 (dPar (pmap p x1 x2 b1 b2) (dBang a1 a2 x1 x2 q)) pf
| lclsL : ∀ p1 p2 q1 q2 a1 a2 b1 b2 s s1 t pf pf1 pf2 g1, let g0 := rm (a1, a2) (rm (b1, b2) g) in
  type_remove s t s1 → In ((a1, a2), vis, Chan lin io t) g → In ((b1, b2), hid, s) g →
  g1 = (((a1, a2), vis, Chan lin empty t) :: ((b1, b2), hid, s) :: g0) →
  lts (((a1, a2), vis, Chan lin out t) :: ((b1, b2), hid, s) :: g0) p1 (aOut a1 a2 b1 b2)
        (((a1, a2), vis, Chan lin empty t) :: ((b1, b2), vis, s1) :: g0) p2 pf1 →
  lts (((a1, a2), vis, Chan lin inp t) :: ((b1, b2), ill, s1) :: g0) q1 (aIn a1 a2 b1 b2)
        (((a1, a2), vis, Chan lin empty t) :: ((b1, b2), vis, s) :: g0) q2 pf2 →
  lts g (dPar p1 q1) (aTau a1 a2) g1 (dNu b1 b2 (dPar p2 q2)) pf
| ucomL : ∀ g1 p1 p2 q1 q2 a1 a2 b1 b2 s s1 t pf pf1 pf2, let g0 := rm (b1, b2) g in
  let g' := ((b1, b2), vis, s1):: g0 in type_remove s t s1 → In ((a1, a2), vis, Chan un io t) g →
  In ((b1, b2), vis, s) g → In ((a1, a2), vis, Chan un io t) g1 → In ((b1, b2), vis, s) g1 →
  lts g p1 (aOut a1 a2 b1 b2) (((b1, b2), vis, s1) :: g0) p2 pf1 →
  lts g' q1 (aIn a1 a2 b1 b2) g1 q2 pf2 → lts g (dPar p1 q1) (aTau a1 a2) g1 (dPar p2 q2) pf
| open : ∀ p a1 a2 b1 b2 g1 p' t pf pf1, In ((b1, b2), hid, t) g →
  lts (((b1, b2), vis, t) :: rm (b1, b2) g) p (aOut a1 a2 b1 b2) g1 p' pf1 →
  lts g (dNu b1 b2 p) (aOut a1 a2 b1 b2) g1 p' pf
| hide : ∀ b1 b2 t p a h1 h2 h' h'' t' p' pf pf1 g1, notHidden b1 b2 a →
g = h1 ++(b1, b2, hid, t) :: h2 →
  lts (h1 ++(b1, b2, vis, t) :: h2) p a (h' ++(b1, b2, vis, t') :: h'') p' pf1 →
  g1 = (h' ++(b1, b2, hid, t') :: h'') → lts g (dNu b1 b2 p) a g1 (dNu b1 b2 p') pf.
(* | uOut ···| lIn ···| uIn ···| lcomL ···| uclsL ··· *)
```

The input cases rely on function `pmap` (*cf.* § 2) to substitute a variable with a name in the continuation process. The replication case `uRin` uses function `newEntries` to build an environment containing only the *hidden* entries that have been *shifted* by function `dep_relabel`. Function `InB` checks if a channel a is mapped by an environment Γ while also returning the entry associated to a, when $a \in \mathsf{dom}(\Gamma)$.

```
Fixpoint newEntries (g : typeEnv) (l : list (name * name)) (max : nat) : typeEnv :=
  match l with | nil ⇒ nil
  | (a1, a2) :: tail ⇒ let (_, notNew) := InB (a1, a2) g in if notNew then newEntries g tail max
  else let (pair, _) := InB (a1, a2 − max) g in
  ((a1, a2), fst pair, snd pair) :: newEntries g tail max end.
```

In the communication cases, *cf.* `lcomL`, `lclsL`, `ucomL`, and `uclsL`, we record the channel that is the subject of the τ-communication. This simplifies the proof of subject reduction, and has no semantic impact (*cf.* [24]). In the inductive cases—the communication cases, `open`, and `hide`—we build the hypotheses environment by manipulating the input environment. For instance, in rule `open` we receive an environment Γ and a process $(\kappa\, b)P$ to obtain hypothesis `In (b, hid, t) g`. We then take as hypothesis an output transition from `((b, vis, t):: rm b g)` and P: since we check typings by using the predicate `In`, the position of the entry in the list is irrelevant. Rule `hide` makes uses of the predicate `notHidden` to verify that the name b hidden in $(\kappa\, b)P$ does not occur in the action, `∀ b c. notHidden b (aTau c)`. Differently to the other rules, `hide` makes uses of `Datatypes.app` to define the structure of the input environment; this is similar to the use of the predicate `In`

(*cf.* `List.in_split`). This format specifies the shape of the environment of the hypothesis, which is key to determine the environment reached by applying the rule.

4 Subject reduction, and other results

We discuss aspects of the mechanisation of subject reduction, outlining also auxiliary results: Barendregt's convention preservation and the extraction of a certified OCaml interpreter. App. A contains a few details of the proof of subject reduction.

We start by setting axiomatic conditions to rule out divergent defining equations of the form $T = p[T]^m$ and $P = u!v.P$.

```
Parameter nonRecTypes: (∀ m p t, t = Chan m p t → False) ∧
  (∀ m m1 m2 p p1 p2 t, Chan m1 p1 t = Chan m p (Chan m2 p2 t) → False).
Parameter nonDivergentProcesses : ∀ g p a g' nd_pf, ¬ lts g p a g' p nd_pf.
```

The first result says that the hidden channels of a process P relabelled w.r.t. a non-duplicated superset of the hidden channels of P does not contain duplicates. Function `List.count_occ` counts the occurrences of pairs of names in a list (*cf.* § 2).

```
Definition incl_ (l m: list (name*name)) := ∀ a:name*name, In a l →
  In a m ∧ count_occ nat_pair_eq_dec l a = count_occ nat_pair_eq_dec m a.
Proposition prop27 : ∀ (p : dproc) (s : list (name*name)) (pf : NoDup s),
  incl_ (secLabs p) s → NoDup (secLabs (dep_relabel p s pf)).
```

The next result relies on a *sanity check* of the hidden channels of process P w.r.t. environment Γ in $\Gamma \triangleright P$. We require that the second entry of a hidden channel of P (1) is positive (*cf.* `sl_pos`), and (2) is less than or equal than the maximum of Γ. Both requirements are essential to preserve the soundness of the relabelling mechanism. We establish that the domain of Δ—the environment returned by function `newEntries` applied to Γ and the hidden channels of the relabelling of P and the maximum of Γ—and the domain of Γ are disjoint, provided that the sanity check on P and Γ holds.

```
Lemma new_entries_disjoint : ∀ g p pf, sl_pos p → max (secLabs p) <= maximum g →
  disjoint (domain g) (domain (newEntries g (secLabs (dep_relabel p (domain g) pf)) (maximum g))).
```

First milestone. The first result clarifies the semantics of our split-free system: if $\Gamma_1 + \Gamma_2 = \Gamma$ and $\Gamma_1 \vdash P$ and $\Gamma \triangleright P \xrightarrow{\alpha} \Gamma' \triangleright P'$ then there is $\mathcal{K}_1$ s.t. $\Gamma_1 \triangleright P \xrightarrow{\alpha} \mathcal{K}_1 \triangleright P'$. That is, a process that (i) is *typed by a subenvironment* Γ_1 of Γ, and (ii) does a transition w.r.t. Γ, can do *the same transition* w.r.t. Γ_1. The proof of `split_lts_exist_lts` is closed by applying `dependent induction` [25] to $\Gamma \triangleright P \xrightarrow{\alpha} \Gamma' \triangleright P'$ while relying on `split_not_exist_lts`.

```
Lemma split_lts_exist_lts : ∀ g1 g2 g p a g' p' pf1 pf2 pf nd_pf nd_pf1,
  tEnv_split g1 g2 g pf1 pf2 pf → typeCheck g1 pf1 p → lts g p a g' p' nd_pf → sl_pos p →
  ∃ k1, lts g1 p a k1 p' nd_pf1.
```

Second milestone. The second milestone consists of a collection of results on typing. The *substitution lemma* states that if $\Gamma, b\colon (\mathtt{f}, S_1), x\colon (\mathtt{v}, S) \vdash P$ and $\mathtt{f} \neq \mathtt{h}$ and $S_1 + S = S_2$ then $\Gamma, b\colon (\mathtt{v}, S_2) \vdash P\{b/x\}$. The requirement `x2 < maximum g ∨ x2 < b2` is needed to maintain the invariant on the maximum of the environment (*cf.* § 2.1).

```
Lemma typeCheck_substitution: ∀ b1 b2 x1 x2 f s1 s s2 g pf pf1 p (nd_pf:NoDup (domain g)),
  In (b1, b2, f, s1) g → In (x1, x2, vis, s) g → (x1,x2)≠ (b1, b2) → (x2 < maximum g ∨ x2 < b2)→
  f ≠ hid → type_plus s1 s s2 → typeCheck g pf p →
  typeCheck ((b1, b2, vis, s2) :: rm (b1, b2) (rm (x1, x2) g)) pf1 (pmap p x1 x2 b1 b2).
```

Another result is that *typing is preserved by relabelling* (*cf.* uRIn of Fig. 3): if $\Gamma \vdash P$ and $\mathsf{rlb}(\Gamma, P) = \mathcal{K}, Q$ then $\mathcal{K}, \Gamma\downarrow_{\mathtt{i}} \vdash Q$. The operator $\Gamma\downarrow_{\mathtt{i}}$, mechanised as `illECast`, casts the polarity of linear hidden and illegal channels (only) to empty.

```
Lemma typeCheck_dep_relabel2: ∀ p g pf nd_pf pf1 k, sl_pos p →
  typeCheck g pf p → k = newEntries g (secLabs (dep_relabel p (domain g) nd_pf)) (maximum g) →
  typeCheck (k ++ illECast g) pf1 (dep_relabel p (domain g) nd_pf).
```

4.1 Subject Reduction Theorem

The design of the typed LTS of Fig. 3 which omits the use of environment splitting (*cf.* Fig. 1), requires a stronger (and harder) result to close subject reduction.

Lemma 1 (Strong subject reduction). *If $\Gamma_1 \vdash P$ and $\Gamma_1 + \Gamma_2 = \Gamma$ and $\Gamma \triangleright P \xrightarrow{\alpha} \mathcal{H} \triangleright P'$ and $\Gamma_1 \triangleright P \xrightarrow{\alpha} \mathcal{K}' \triangleright P'$ then $\mathcal{K}' \vdash P'$.*

The need for Lemma 1 comes from the combination of the typing (*cf.* tPar, Fig. 2) and LTS (*cf.* ParL, Fig. 3) rules for parallel composition. Consider the parallel case, $P = P_1 \parallel Q_1$, for Theorem 1. When apply induction, the goal is $\Gamma' \vdash P_2 \parallel Q_1$, where $\Gamma \triangleright P_1 \xrightarrow{\alpha} \Gamma' \triangleright P_2$ with the following IH: if $\Gamma \vdash P_1$ then $\Gamma' \vdash P_2$. The main complication is that, in general, $\Gamma \not\vdash P_1$. By inversion of $\Gamma \vdash P_1 \parallel Q_1$, we indeed obtain $\Gamma_1 \vdash P_1$ and $\Gamma_2 \vdash P_2$ and $\Gamma_1 + \Gamma_2 = \Gamma$ which is insufficient to complete the proof. Instead, the fine-grained formulation of Lemma 1 allows us to prove the desired result and obtain Theorem 1 as a byproduct. The outline of the mechanisation of Lemma 1 is in App. A; its proof takes 16k loc.

Theorem 1 (Subject reduction). *If $\Gamma \vdash P$ and $\Gamma \triangleright P \xrightarrow{\alpha} \Gamma' \triangleright P'$ then $\Gamma' \vdash P'$.*

4.2 Other results

Barendregt's convention (*BC*) preservation. The convention is implemented by `wf`. Typed processes satisfy BC: this result and subject reduction enforce preservation.

```
Definition wf (p : dproc) : Prop := NoDup (secLabs p) ∧ disjoint (fn p) (secLabs p).
Corollary wf_preserve g1 p1 a g2 p2 pf nd_pf: sl_pos p1 → typeCheck g1 pf p1 →
  lts g1 p1 a g2 p2 nd_pf → wf p2.
```

LTS typing avoids cases such as $\Gamma \triangleright P \xrightarrow{a?b} \Gamma' \triangleright P'$ where $\mathsf{fn}(P') \cap \mathsf{hid}(P') = \emptyset$ does not hold. *E.g.*, Rule uRIn of Fig. 3 would rule this out when $P = *a?x.x!v.\mathsf{nil} \parallel (\kappa\, b)Q$ and $b \in \mathsf{fn}(P') \wedge b \in \mathsf{hid}(P')$ since it requires the flag of channel b *not* to be hid.

Extraction of a certified interpreter. The LTS of § 3 can be deployed soundly in mainstream programming languages. Function `elts` receives a type environment Γ, a process P, an action α, and other optional arguments, and returns (i) a pair (Γ', P'), or (ii) none: in case (i), we show that $\Gamma \triangleright P \xrightarrow{\alpha} \Gamma' \triangleright P'$: that is, the interpreter is sound. The code is extracted to OCaml and contains less than fifty types/values.

5 Conclusions

We present an LTS semantics for the linear π-calculus that lends itself well to mechanically-checked software. We defend that the semantics can be used as a basis for building automated static analyses of linear π-calculus system models that is equipped with mechanisms to report channel names that are determined to be the source of a defect. The certified runnable code extracted from the mechanisation permits the study of elaborate examples (that go beyond small pen-and-paper inspections) via simulations, implementing Klein *et al.*'s "run your research" challenge [23].

Approaches to mechanising the π-calculus. There are various different ways of dealing with name binding and scope extrusion in mobile process calculi. They may take an axiomatic approach, building on theories to deal with (capture avoiding) α-conversion (inspired by seminal work on mechanising the λ-calculus [9]), or they may be based on logical techniques.

The axiomatic approach can be either first-order, *e.g.,* [26,16,11], or higher-order, *e.g.,* [22,34,33,1,44], via the Higher-Order Abstract Syntax or/and the "Theory of Contexts". The main limitations of these approaches are the cumbersome and heavy representation of bound names, not being preserved by the semantics of the calculus, and the existence of "exotic" terms and the extra work needed to exclude them.

The logical approaches are based on either Nominal Logics [31,2] or on the Concurrent Logical Framework (CLF) [39,5,37]. Proponents of the former argue that formal proofs become similar to those in books by simplifying the reasoning about bound names and allowing processes to be equated up-to α-conversion. However, Nominal Logics require equivariance lemmata, stating that name permutations are preserved by the operations of the static and dynamic semantics: static analyses based on such a framework would find it hard to refer to a specific bound name working up-to α-conversion. CFL uses a special quantifier (∇) for encoding both judgements and fixed points. It provides a formal treatment of bound names and highlights the differences between the various classical definitions of bisimulation via the interplay of quantifiers. Although its focus is on proof systems, proof search, and modal characterisations of equivalences, it has been used to show type safety of a behavioural type system for the π-calculus [40]. It is however not clear if algorithmic type checking/inference or code extraction are possible.

There are several works [28,20,21,22,34,32,33,1,2,29,30] that mechanise semantic theories of the π-calculus to prove type preservation and safety results [19,11,6,10,45]. These works however use reduction semantics, not LTSs. The type systems presented are declarative and, by admission of the authors, "turning" them into type checking and/or type inference algorithms is an added challenge. In our case, the type checking algorithm can be derived from the rules by adding a return environment containing the capabilities not consumed by parallel processes, following [12,13]. Type inference can be obtained by indicating which channels are meant to be linear (*cf.* [14]). None of these works feature the extraction of an interpreter from the semantic mechanisation. This is relatively straightforward in our case, because we designed the LTS with this intent.

Concurrent Benchmark Challenges [3].. We address all challenges (one partially):

Linearity and Behavioural Type Systems: we define a π-calculus with linear types and prove type preservation (*i.e.,* subject-reduction). Instead of using the more straightforward approach with reduction semantics, we used an LTS-based semantics which is more compositional. Our approach is original—it uses neither α nor structural congruence—and features an important novelty, a split-free typed LTS.

Name Passing and Scope Extrusion: type environments have information on name visibility and usage permission, dynamically managing linearity via capabilities updates due to scope intrusion/extrusion, while also dealing soundly with binding.

Coinduction and Infinite Processes: the minimal calculus we use has infinite processes via replicated inputs. Using the relabelling mechanism, the LTS takes care of incremental name generation (as showing in Sec. 4). Work on a behavioural equivalence is in progress; the mechanisation of that is left for future work.

Reusable mechanisms. We develop an innovative, simple yet general treatment of bound names, capabilities, and type environment operations for substructural type systems and labelled transition systems. We contend that the constructions that are reusable in other contexts (program calculi) are the following. 1. The treatment of binders leading to an α-congruence free calculus that preserve Barendregt's convention for typed processes. 2. Type environments with visible and hidden (scoped) entries, internalising identifier observability and usage permission. 3. Substructural manipulation of resource policies with an algebra on types and an associated split-free semantics for the language. 4. A general soundness theorem leading to the extraction of a certified interpreter.

Acknowledgements

This work was partly supported by the DReason project (2024.15569.PEX, https://doi.org/10.54499/2024.15569.PEX), NOVA LINCS (UID/04516/2025) with the financial support of FCT.IP (https://doi.org/10.54499/UID/04516/2025) and the UOM Research Seed Fund CPSRP01-26 (Runtime Verification and Adaptation for Industry 5.0).

A Appendix

In this section we provide more details about the proof of subject reduction, including important lemmas that are not presented in § 4. The mechanisation is available at https://github.com/marcogiunti/Linear-Pi-Calculus. The repository contains approximately 65k SLOC.

We start by establishing useful properties of environment splitting (*cf.* Fig. 1), that is type `tEnv_split`. We show that the three environments related by the relation have the same domain.

```
Lemma split_domain: ∀ g1 g2 g pf1 pf2 pf x, tEnv_split g1 g2 g pf1 pf2 pf → (In x (domain g1) ↔
  In x (domain g2)) ∧ (In x (domain g1) ↔ In x (domain g)) ∧ (In x (domain g2) ↔ In x (domain g3)).
```

An important fact is the existence of a *neutral environment* that is consumed. Lemma `split_exist_neutral` is crucial to close the proof of subject reduction (*cf.* Theor. 1).

```
Lemma split_exist_neutral : ∀ g pf (nd_pf : NoDup(domain g)),
  ∃ neutral pfn (nd_pfn : NoDup (domain neutral)), tEnv_split g neutral g pf pfn pf.
Lemma neutral_is_cons: ∀ g neutral pf pfn pf1, tEnv_split g neutral g pf pfn pf1 → cons neutral pfn.
```

The next lemma enables a form of compositionality of environment splitting that is paramount to prove our first milestone (*cf.* next paragraph).

```
Lemma split_of_split : ∀ g0 w0 g1 g2 g pf0 pfw0 pf1 pf2 pf (nd_pf1 : NoDup (domain g1)),
  tEnv_split g0 w0 g1 pf0 pfw0 pf1 → tEnv_split g1 g2 g pf1 pf2 pf →
  ∃ w2 pfw2, tEnv_split g0 w2 g pf0 pfw2 pf.
```

We then analyse properties of the transition system `lts`. The cases spawned by actions of replicated input prefixes are particularly challenging. We define an inductive notion of `replication` to collect more information on such processes. The result below says that unrestricted input actions of a process P (i) maintain the same environment's domain *or* (ii) add a new disjoint environment. Cases (i) and (ii) depict a transition spawned by an input process, and by a replicated input process, respectively. In case (ii), the disjoint environment is obtained from the relabelling of the replicated subprocess Q of P spawning the transition, that is `replication p p1 q`, and from the environment Γ.

```
Lemma lts_un_input_action_newEntries: ∀ g p a1 a2 b1 b2 r g1 p1 nd_pf pf pol t k, sl_pos p →
  NoDup (secLabs p) → lts g p (aIn a1 a2 b1 b2) g1 p1 nd_pf →
  g1 = k ++(b1, b2, vis, r) :: rm(b1, b2) g → typeOf g (a1, a2) (Chan un pol t) →
  disjoint (domain k) (domain ((b1, b2, vis, r) :: rm(b1, b2) g)) → k = nil ∨
  (∃ q, replication p p1 q ∧ incl (secLabs q) (secLabs p) ∧
  k = newEntries g (secLabs (dep_relabel q (domain g) pf)) (max (domain g))).
```

To attack the first milestone, we first analyse under which conditions a process P doing a transition α w.r.t. a subenvironment Γ_1 cannot do the same transition w.r.t. the super environment Γ, where $\Gamma_1 + \Gamma_2 = \Gamma$; the predicate `wf` implements the Barendregt's convention and is discussed at the end of § 4. Four cases arises: (1) $\alpha = x?y$ and $\Gamma_1(x) = (\mathtt{f}_1, \downarrow [T]^\iota)$ and $\Gamma(x) = (\mathtt{f}, \updownarrow [T]^\iota)$; (2) $\alpha = x?y$ and $\Gamma_1(x) = (\mathtt{f}_1, p[T]^m)$ and $\Gamma(y) = (\mathtt{f}_2, T_y)$ and $T + T_y$ is undefined; (3) $\alpha = x?y$ and $\Gamma_1(x) = (\mathtt{f}, p[T]^m)$ and $\Gamma_1(y) = (\mathtt{i}, T_y)$ and $\Gamma(y) = (\mathtt{h}, T'_y)$; (4) $\alpha = x!y$ and $\Gamma_1(x) = (\mathtt{f}_1, \uparrow [T]^\iota)$ and $\Gamma(x) = (\mathtt{f}, \updownarrow [T]^\iota)$.

```
Lemma split_not_exist_lts : ∀ g1 g2 g k1 p a  p' pf1 pf2 pf nd_pf1 (nd_pf2 : NoDup (domain g2)) nd_pf,
  tEnv_split g1 g2 g pf1 pf2 pf → lts g1 p a k1 p' nd_pf1 → NoDup (secLabs p) →
  incl (secLabs p) (domain (hid_proj g1)) → wf p → sl_pos p → max (secLabs p) <= maximum g1 →
  (¬ ∃ g', lts g p a g' p' nd_pf) → ∃ x1 x2 y1 y2 t,
  (a = aIn x1 x2 y1 y2 ∧ typeOf g1 (x1, x2) (Chan lin inp t) ∧ typeOf g (x1, x2) (Chan lin io t)) ∨
  ((a = aIn x1 x2 y1 y2 ∧ ∃ m p ty, typeOf g1 (x1, x2) (Chan m p t) ∧ typeOf g (y1, y2) ty ∧
  ¬ ∃ t1, type_plus t ty t1)) ∨
  ((a = aIn x1 x2 y1 y2 ∧ flagOf g1 (y1, y2) ill ∧ flagOf g (y1, y2) hid)) ∨
  (a = aOut x1 x2 y1 y2 ∧ typeOf g1 (x1, x2) (Chan lin out t) ∧ typeOf g (x1, x2) (Chan lin io t)).
```

A.1 Mechanisation of Subject Reduction

The mechanisation of Lemma 1 (*cf.* § 4) is below; its proof takes 16k loc. As discussed before, function `sl_pos` is used to enforce the soundness of relabelling.

```
Lemma sr_strong : ∀ g p a p' nd_pf pf g1 g2 k1 h pf1 pf2 pfk1 nd_pf1, sl_pos p → typeCheck g1 pf1 p →
  tEnv_split g1 g2 g pf1 pf2 pf → lts g p a h p' nd_pf → lts g1 p a k1 p' nd_pf1 →
  typeCheck k1 pfk1 p'.
```

The proof is done by applying `dependent induction` to $\Gamma \triangleright P \xrightarrow{\alpha} \mathcal{H} \triangleright P'$ (*cf.* Lemma 1). We discuss a few selected cases.

ParL This is the more complex case. We have $P = P_1 \,\|\, P_3$ and $\Gamma \triangleright P_1 \,\|\, P_2 \xrightarrow{\alpha} \mathcal{K}_1 \triangleright P_2 \,\|\, P_3$. The goal is $\mathcal{K}_1 \vdash P_2 \,\|\, P_3$. By inversion of $\Gamma_1 \vdash P$, we obtain $\Gamma_0 + \Gamma_3 = \Gamma_1$ and $\Gamma_0 \vdash P_1$ and $\Gamma_3 \vdash P_3$.

First, we perform `eapply split_lts_exist_lts` to $\Gamma_0 + \Gamma_3 = \Gamma_1$. We obtain $\Gamma_0 \triangleright P_1 \xrightarrow{\alpha} \mathcal{K}_0 \triangleright P_2$ with $\mathcal{K}_0$ fresh. We then apply `split_of_split` to $\Gamma_0 + \Gamma_3 = \Gamma_1$ and obtain $\Gamma_0 + \mathcal{W}_2 = \Gamma$ with $\mathcal{W}_2$ fresh. We instantiate the I.H. with these results and deduce: $\mathcal{K}_0 \vdash P_2$.

Second, we perform `destruct a`: that is, we do a case analysis on the action α.

In the **input** case $\alpha = a?b$, that is a is (now) the subject and b is the payload, we first `destruct` the entry $\Gamma_0(b) = (\mathtt{f}, T)$, and then `destruct` the entry $\Gamma_0(a) = (\mathtt{v}, p'[S]^{m'})$; finally we perform `eapply TPar`. The case $T = \mathsf{bt}$ is simpler and we omit it. Otherwise, we have $T = p[T']^m$ and we continue by destructing m. In both cases $m = \iota$ and $m = \omega$ we proceed by destructing the flag $\mathtt{f}$. We have two subcases corresponding to $\mathtt{f} = \mathtt{i}$ and $\mathtt{f} = \mathtt{v}$.

In the **linear illegal payload** case $\Gamma_0(b) = (\mathtt{i}, p[T]^\iota)$, we deduce that b is not used in P_1 (*cf.* $\Gamma_0 \vdash P_1$) and in turn that $p = \emptyset$. After destructing the type of the subject a, that is $p'[S]^{m'}$, and in turn m', we start with the **linear subject** case $\Gamma_0(a) = (\mathtt{v}, \downarrow [S]^\iota)$. To do `eapply TPar`, we provide the environments $\mathcal{K}_0$ and Γ_3', where $\Gamma_3' = \Gamma_3 \backslash b, b\colon (\mathtt{v}, \emptyset[T]^\iota)$. We thus need to prove that $\mathcal{K}_0 + \Gamma_3' = \mathcal{K}_1$ and $\mathcal{K}_0 \vdash P_2$ and $\Gamma_3' \vdash P_3$. The first result is proved by using the hypothesis $\Gamma_0 = \Gamma_3 = \Gamma_1$ and by doing a syntactic analysis on the entries that differ in each environment; the second result is provided by the I.H.; the last result is obtained by applying a lemma allowing to change the flag `i` with `v`. The **unrestricted subject** case $\Gamma_0(a) = (\mathtt{v}, p[S]^\omega)$ is more involved, because the input action can spawn from a input, or from a replicated input. We proceed by distinguishing the two cases by means of a syntactical analysis of $\mathcal{K}_0$. The most interesting case is *replicated input*. In such case we know that $\mathcal{K}_0$ is of the form `ne0 ++(b1, b2, vis, r) :: rm(b1, b2) g0` where `ne0` is obtained by applying `newEntries` to Γ_0 and to the hidden channels of the relabelling R of the subprocess of P_1 spawning the replication. Moreover, $\mathcal{K}_1$ is of the form `ne1 ++(b1, b2, vis, r') :: rm(b1, b2) g1` where `ne1` is obtained from applying `newEntries` to Γ_1 and to the relabelling R. We do `eapply TPar` by passing the environments $\mathcal{K}_0$ and $\mathcal{K}_3$, where $\mathcal{K}_3$ is of the form `ne3 ++(b1, b2, vis, Chan lin empty tb) :: rm(b1, b2) g3`, and `ne3` is obtained from applying `newEntries` to Γ_3 and to the relabelling R. We conclude this subcase by proving that $\mathcal{K}_0 + \mathcal{K}_3 = \mathcal{K}_1$ and $\mathcal{K}_0 \vdash P_2$ and $\mathcal{K}_3 \vdash P_3$. The first result is obtained by using a lemma to distribute splitting, and a lemma for splitting new entries. The second result is provided by the I.H., while the last result is obtained by weakening.

The **linear visible payload** case $\Gamma_0(b) = (\mathtt{v}, p[T]^\iota)$ is similar. We need to consider that p can be any capability compatible with the payload S, given that by hypothesis $p[T]^\iota + S$ is defined.

The last channel subcase is the **unrestricted payload** case $\Gamma_0(b) = (\mathtt{f}, p[T]^\omega)$, where $p[T]^\omega + S$ can increase the capability of b. Here we need to keep attention to environment splitting (*cf.* Fig. 1) because only splitting with the same unrestricted capabilities is allowed. If $p[T]^\omega + S \neq p[T]^\omega$ then we have to weaken the typed assumptions by adding the required missing unrestricted capabilities. We refer to the for the details of this subcase. In total, the input case requires 3k loc.

In the **output** case $\alpha = a!b$, we first analyse the entry $\Gamma(b) = (\mathtt{f}, T)$, and then destruct $\Gamma(a) = (\mathtt{v}, p'[S]^{m'})$ and `apply TPar`, where $\mathtt{f} = \mathtt{v}$ or $\mathtt{f} = \mathtt{h}$. The output case is substantially simpler than the input case, and require a couple of hundred lines. The complexity of the input case has a great impact on the analysis of the **tau** case $\alpha = \tau$, and in particular when `a = aTau c1 c2` with $\Gamma(c)$ unrestricted, because of the replication scenario. The proof of the tau case takes 7k loc; we omit all details.

uRIn The proof of this case follows the reasoning outlined in case TPar, subcase $\alpha = u?b$ with $\Gamma(a) = (\mathtt{v}, p[S]^{\omega})$, although this case is quite simpler. For what concerns the shape of environments, most of the reasoning used in the referred case of TPar is guided by the syntactic analysis of the redex environments, and still stands here. In addition, in this case we have to build a type derivation for the redex: to this aim, we use results that allow to type check the relabelled process in the continuation.

lClsL This case follows by induction. We have $P = P_1 \parallel Q_1$ and $\Gamma \triangleright P_1 \parallel Q_1 \xrightarrow{\tau} \mathcal{K}_1 \triangleright (\kappa\, b)(P_2 \parallel Q_2)$. We note that $\mathcal{K}_1 = \Gamma_1 \backslash_a, a\colon (\mathtt{v}, \emptyset[T]^{\iota})$ and $\Gamma_1(b) = (\mathtt{h}, S_0)$, for some type S_0. The goal is $\mathcal{K}_1 \vdash (\kappa\, b)(P_2 \parallel Q_2)$.

By inversion of $\Gamma_1 \vdash P$, we obtain $\Gamma_3 + \Gamma_4 = \Gamma_1$ and $\Gamma_3 \vdash P_1$ and $\Gamma_4 \vdash Q_1$. The first I.H. (*cf.* P_1) is

(IH1) $\quad \forall \mathcal{K}'\, \mathcal{K}_2 \,.\, \Gamma_3 + \mathcal{K}_2 = \Gamma \backslash_a \backslash_b, a\colon (\mathtt{v}, \uparrow [T]^{\iota}), b\colon (\mathtt{h}, S) \Rightarrow \Gamma_3 \triangleright P_1 \xrightarrow{a!b} \mathcal{K}' \triangleright P_2 \Rightarrow \mathcal{K}' \vdash P_2$

We instantiate (IH1) with $\mathcal{K}' = \Gamma_3 \backslash_a \backslash_b, a\colon (\mathtt{v}, \emptyset[T]^{\iota}), b\colon (\mathtt{h}, S_3)$, where $T_3^b - T = S_3$ and $\Gamma_3(b) = (\mathtt{h}, T_3^b)$. Next, we find Γ_2 s.t. $\Gamma_3 + \Gamma_2 = \Gamma \backslash_a \backslash_b, a\colon (\mathtt{v}, \uparrow [T]^{\iota}), b\colon (\mathtt{h}, S)$ by relying on `split_of_split`, and instantiate (IH1) with $\mathcal{K}_2 = \Gamma_2$. We thus conclude: $\Gamma_3 \backslash_a \backslash_b, a\colon (\mathtt{v}, \emptyset[T]^{\iota}), b\colon (\mathtt{h}, S_3) \vdash P_2$. We proceed similarly with the second I.H. (*cf.* P_2) and obtain: $\Gamma_4 \backslash_a \backslash_b, a\colon (\mathtt{v}, \emptyset[T]^{\iota}), b\colon (\mathtt{v}, S_4) \vdash Q_2$ where $T_4^b + T = S_4$ and $\Gamma_4(b) = (\mathtt{i}, T_4^b)$.

Remember the goal $\Gamma_1 \backslash_a, a\colon (\mathtt{v}, \emptyset[T]^{\iota}) \vdash (\kappa\, b)(P_2 \parallel Q_2)$ with $\Gamma_1(b) = (\mathtt{h}, S_0)$. We do `eapply TNew` and obtain the new goal: $\Gamma_1 \backslash_a \backslash_b, a\colon (\mathtt{v}, \emptyset[T]^{\iota}), b\colon (\mathtt{v}, S_0) \vdash P_2 \parallel Q_2$.

Finally, we do `eapply TPar` by passing $\Gamma_3 \backslash_a \backslash_b, a\colon (\mathtt{v}, \emptyset[T]^{\iota}), b\colon (\mathtt{v}, S_3)$ and $\Gamma_4 \backslash_a \backslash_b$, $a\colon (\mathtt{v}, \emptyset[T]^{\iota})$, $b\colon (\mathtt{v}, S_4)$ and get the last goal:

$$\Gamma_3 \backslash_a \backslash_b, a\colon (\mathtt{v}, \emptyset[T]^{\iota}), b\colon (\mathtt{v}, S_3) + \Gamma_4 \backslash_a \backslash_b, a\colon (\mathtt{v}, \emptyset[T]^{\iota}), b\colon (\mathtt{v}, S_4) = \Gamma_1 \backslash_a \backslash_b, a\colon (\mathtt{v}, \emptyset[T]^{\iota}), b\colon (\mathtt{v}, S_0)$$

The result is proved by relying on the hypothesis $\Gamma_3 + \Gamma_4 = \Gamma_1$, and by showing that $S_3 + S_4 = S_0$.

Hide This case marks the end of the proof and relies on induction and on a careful syntactic manipulation of the structure of the environments, which eventually uses types and results in `Sorting.Permutation`.

Standard Subject Reduction. The mechanisation and proof of Theorem 1 is below.

```
Theorem sr : ∀ g p a g' p' nd_pf pf pf', sl_pos p → typeCheck g pf p → lts g p a g' p' nd_pf →
  typeCheck g' pf' p'.
Proof. intros. assert (K := nd_pf). apply split_exist_neutral with g pf in K.
  destruct K as [neutral [pfn [nd_pfn TS]]]. eapply sr_strong in H. all: eauto.
Qed.
```

To close the result, we apply `split_exist_neutral` to Γ and find $\mathcal{N}$ s.t. $\Gamma + \mathcal{N} = \Gamma$. The instantiation of `sr_strong` with $\Gamma_1 = \Gamma$ and $\Gamma_2 = \mathcal{N}$ and $\mathcal{H} = \Gamma'$ and $\mathcal{K}_1 = \Gamma'$ allows us to close the proof.

References

1. Reynald Affeldt and Naoki Kobayashi. A Coq library for verification of concurrent programs. In Carsten Schürmann, editor, *Proceedings of the Fourth International Workshop on Logical Frameworks and Meta-Languages, LFM@IJCAR 2004*, volume 199 of *Electronic Notes in Theoretical Computer Science*, pages 17–32. Elsevier, 2004. doi:10.1016/J.ENTCS.2007.11.010.
2. Jesper Bengtson and Joachim Parrow. Formalising the pi-calculus using nominal logic. *Logical Methods in Computer Science*, 5(2), 2009. URL: http://arxiv.org/abs/0809.3960.
3. Marco Carbone, David Castro-Perez, Francisco Ferreira, Lorenzo Gheri, Frederik Krogsdal Jacobsen, Alberto Momigliano, Luca Padovani, Alceste Scalas, Dawit Legesse Tirore, Martin Vassor, Nobuko Yoshida, and Daniel Zackon. The concurrent calculi formalisation benchmark. In Ilaria Castellani and Francesco Tiezzi, editors, *Coordination Models and Languages - 26th IFIP WG 6.1 International Conference, COORDINATION 2024*, volume 14676 of *Lecture Notes in Computer Science*, pages 149–158. Springer, 2024. doi:10.1007/978-3-031-62697-5_9.
4. Elias Castegren and Tobias Wrigstad. OOlong: A concurrent object calculus for extensibility and reuse. *SIGAPP Appl. Comput. Rev.*, 18(4):47–60, January 2019. doi:10.1145/3307624.3307629.
5. Iliano Cervesato, Frank Pfenning, David Walker, and Kevin Watkins. A Concurrent Logical Framework II: Examples and Applications. Technical report, Carnegie Mellon University, 3 2007. doi:10.1184/R1/6587294.v1.
6. Luca Ciccone and Luca Padovani. A dependently typed linear π-calculus in Agda. In *22nd Int. Symp. on Principles and Practice of Declarative Programming, PPDP 2020*, pages 8:1–8:14. ACM, 2020. doi:10.1145/3414080.3414109.
7. Ernesto Copello, Nora Szasz, and Álvaro Tasistro. Formalisation in constructive type theory of Barendregt's variable convention for generic structures with binders. In Frédéric Blanqui and Giselle Reis, editors, *13th International Workshop on Logical Frameworks and Meta-Languages: Theory and Practice, LFMTP@FSCD 2018*, volume 274 of *EPTCS*, pages 11–26, 2018. doi:10.4204/EPTCS.274.2.
8. Ornela Dardha, Elena Giachino, and Davide Sangiorgi. Session types revisited. *Information and Computation*, 256:253–286, 2017. doi:10.1016/j.ic.2017.06.002.
9. N. J. de Bruijn. Lambda calculus notation with nameless dummies. *Indagationes Mathematicae*, 75(5):381–392, 1972.
10. Joëlle Despeyroux. A higher-order specification of the pi-calculus. In Jan van Leeuwen, Osamu Watanabe, Masami Hagiya, Peter D. Mosses, and Takayasu Ito, editors, *Theoretical Computer Science, Exploring New Frontiers of Theoretical Informatics, IFIP TCS 2000*, volume 1872 of *Lecture Notes in Computer Science*, pages 425–439. Springer, 2000. doi:10.1007/3-540-44929-9_30.
11. Simon J. Gay. A framework for the formalisation of pi-calculus type systems in Isabelle/HOL. In *14th International Conference on Theorem Proving in Higher Order Logics, TPHOLs 2000*, volume 2152 of *Lecture Notes in Computer Science*, pages 217–232. Springer, 2001. doi:10.1007/3-540-44755-5_16.
12. Simon J. Gay and Malcolm Hole. Subtyping for session types in the pi calculus. *Acta Informatica*, 42(2-3):191–225, 2005. doi:10.1007/S00236-005-0177-Z.
13. Marco Giunti. Algorithmic type checking for a pi-calculus with name matching and session types. *J. Log. Algebraic Methods Program.*, 82(8):263–281, 2013. doi:10.1016/J.JLAP.2013.05.003.
14. Marco Giunti. GoPi: Compiling linear and static channels in Go. In Simon Bliudze and Laura Bocchi, editors, *Coordination Models and Languages - 22nd IFIP WG 6.1 International*

Conference, COORDINATION 2020, volume 12134 of *Lecture Notes in Computer Science*, pages 137–152. Springer, 2020. doi:10.1007/978-3-030-50029-0_9.

15. Marco Giunti and Vasco Thudichum Vasconcelos. Linearity, session types and the pi calculus. *Math. Struct. Comput. Sci.*, 26(2):206–237, 2016. doi:10.1017/S0960129514000176.
16. Andrew D. Gordon and Thomas F. Melham. Five axioms of alpha-conversion. In *9th International Conference on Theorem Proving in Higher Order Logics, TPHOLs 1996*, volume 1125 of *Lecture Notes in Computer Science*, pages 173–190. Springer, 1996. doi:10.1007/BFb0105404.
17. Matthew Hennessy. *A distributed Pi-calculus*. Cambridge University Press, 2007.
18. Matthew Hennessy and Julian Rathke. Typed behavioural equivalences for processes in the presence of subtyping. *Math. Struct. Comput. Sci.*, 14(5):651–684, 2004. doi:10.1017/S0960129504004281.
19. Loïc Henry-Gréard. Proof of the subject reduction property for a π-calculus in Coq. Technical Report RR-3698, INRIA, May 1999. URL: https://inria.hal.science/inria-00072970.
20. Daniel Hirschkoff. A full formalisation of pi-calculus theory in the calculus of constructions. In *10th International Conference on Theorem Proving in Higher Order Logics, TPHOLs 1997*, volume 1275 of *Lecture Notes in Computer Science*, pages 153–169. Springer, 1997. doi:10.1007/BFb0028392.
21. Daniel Hirschkoff. *Mise en Oevre de Preuves de Bisimulation*. PhD thesis, École Nationale des Pont et Chausseées, 1999.
22. Furio Honsell, Marino Miculan, and Ivan Scagnetto. Pi-calculus in (co)inductive-type theory. *Theorectical Computer Science*, 253(2):239–285, 2001. doi:10.1016/S0304-3975(00)00095-5.
23. Casey Klein, John Clements, Christos Dimoulas, Carl Eastlund, Matthias Felleisen, Matthew Flatt, Jay A. McCarthy, Jon Rafkind, Sam Tobin-Hochstadt, and Robert Bruce Findler. Run your research: on the effectiveness of lightweight mechanization. In John Field and Michael Hicks, editors, *39th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2012*, pages 285–296. ACM, 2012. doi:10.1145/2103656.2103691.
24. Naoki Kobayashi, Benjamin C. Pierce, and David N. Turner. Linearity and the pi-calculus. *ACM Trans. Program. Lang. Syst.*, 21(5):914–947, 1999. doi:10.1145/330249.330251.
25. Conor McBride. Elimination with a motive. In Paul Callaghan, Zhaohui Luo, James McKinna, and Robert Pollack, editors, *Types for Proofs and Programs, International Workshop, TYPES 2000, Selected Papers*, volume 2277 of *Lecture Notes in Computer Science*, pages 197–216. Springer, 2000. doi:10.1007/3-540-45842-5_13.
26. Thomas F. Melham. A mechanized theory of the pi-calculus in HOL. *Nordic Journal of Computing*, 1(1):50–76, 1994.
27. Robin Milner. *Communicating and mobile systems - the Pi-calculus*. Cambridge University Press, 1999.
28. Otmane Aït Mohamed. Mechanizing a pi-calculus equivalence in HOL. In *10th International Conference on Theorem Proving in Higher Order Logics, TPHOLs 1995*, volume 971 of *Lecture Notes in Computer Science*, pages 1–16. Springer, 1995. doi:10.1007/3-540-60275-5_53.
29. Dominic A. Orchard and Nobuko Yoshida. Using session types as an effect system. In *8th International Workshop on Programming Language Approaches to Concurrency- and Communication-cEntric Software, PLACES 2015*, volume 203 of *EPTCS*, pages 1–13, 2015. doi:10.4204/EPTCS.203.1.
30. Roly Perera and James Cheney. Proof-relevant pi-calculus: a constructive account of concurrency and causality. *Mathematical Structures in Computer Science*, 28(9):1541–1577, 2018. doi:10.1017/S096012951700010X.

31. Andrew M. Pitts. Nominal logic, a first order theory of names and binding. *Inf. Comput.*, 186(2):165–193, 2003. doi:10.1016/S0890-5401(03)00138-X.
32. Christine Röckl. A first-order syntax for the pi-calculus in Isabelle/HOL using permutations. *Electronic Notes in Theorectical Computer Science*, 58(1):1–17, 2001. doi:10.1016/S1571-0661(04)00276-2.
33. Christine Röckl and Daniel Hirschkoff. A fully adequate shallow embedding of the [pi]-calculus in Isabelle/HOL with mechanized syntax analysis. *Journal of Functional Programming*, 13(2):415–451, 2003. doi:10.1017/S0956796802004653.
34. Christine Röckl, Daniel Hirschkoff, and Stefan Berghofer. Higher-order abstract syntax with induction in Isabelle/HOL: Formalizing the pi-calculus and mechanizing the theory of contexts. In *4th International Conference on Foundations of Software Science and Computation Structures, FOSSACS 2001*, volume 2030 of *Lecture Notes in Computer Science*, pages 364–378. Springer, 2001. doi:10.1007/3-540-45315-6_24.
35. Davide Sangiorgi and David Walker. *The Pi-Calculus - a theory of mobile processes*. Cambridge University Press, 2001.
36. Chuta Sano, Ryan Kavanagh, and Brigitte Pientka. Mechanizing session-types using a structural view: Enforcing linearity without linearity. *Proc. ACM Program. Lang.*, 7(OOPSLA2):374–399, 2023. doi:10.1145/3622810.
37. Alwen Tiu and Dale Miller. Proof search specifications of bisimulation and modal logics for the π-calculus. *ACM Trans. Comput. Logic*, 11(2), January 2010. doi:10.1145/1656242.1656248.
38. Christian Urban and Michael Norrish. A formal treatment of the Barendregt variable convention in rule inductions. In Randy Pollack, editor, *ACM SIGPLAN International Conference on Functional Programming, Workshop on Mechanized reasoning about languages with variable binding, MERLIN 2005*, pages 25–32. ACM, 2005. doi:10.1145/1088454.1088458.
39. Kevin Watkins, Iliano Cervesato, Frank Pfenning, and David Walker. A concurrent logical framework: The propositional fragment. In Stefano Berardi, Mario Coppo, and Ferruccio Damiani, editors, *Types for Proofs and Programs, International Workshop, TYPES'03, Revised Selected Papers*, volume 3085 of *Lecture Notes in Computer Science*, pages 355–377. Springer, 2003. doi:10.1007/978-3-540-24849-1_23.
40. Kevin Watkins, Iliano Cervesato, Frank Pfenning, and David Walker. Specifying properties of concurrent computations in CLF. *Electronic Notes in Theoretical Computer Science*, 199:67–87, 2008. Proc. Fourth Int. Workshop on Logical Frameworks and Meta-Languages (LFM 2004). doi:10.1016/j.entcs.2007.11.013.
41. Nobuko Yoshida, Martin Berger, and Kohei Honda. Strong normalisation in the pi-calculus. In *16th Annual IEEE Symposium on Logic in Computer Science, LICS 2001*, pages 311–322. IEEE Computer Society, 2001. doi:10.1109/LICS.2001.932507.
42. Nobuko Yoshida, Kohei Honda, and Martin Berger. Linearity and bisimulation. *J. Log. Algebraic Methods Program.*, 72(2):207–238, 2007.
43. Daniel Zackon, Chuta Sano, Alberto Momigliano, and Brigitte Pientka. Split decisions: Explicit contexts for substructural languages. In Kathrin Stark, Amin Timany, Sandrine Blazy, and Nicolas Tabareau, editors, *14th ACM SIGPLAN International Conference on Certified Programs and Proofs, CPP 2025*, pages 257–271. ACM, 2025. doi:10.1145/3703595.3705888.
44. Uma Zalakain. Type-checking session-typed π-calculus with Coq. Master's thesis, University of Glasgow, 2019. URL: https://umazalakain.info/static/Z19/msc-thesis.pdf.
45. Uma Zalakain and Ornela Dardha. π with leftovers: A mechanisation in Agda. In Kirstin Peters and Tim A. C. Willemse, editors, *Formal Techniques for Distributed Objects, Components, and Systems, FORTE 2021*, volume 12719 of *Lect. Notes Comput. Sci.*, pages 157–174. Springer, 2021. doi:10.1007/978-3-030-78089-0_9.